

Informe corto del proyecto Tetris

1. Relación entre las estructuras utilizadas

En el proyecto se utilizaron diferentes estructuras de datos, y cada una tiene una función específica dentro del juego.

El tablero se representa mediante una lista doblemente enlazada de filas. Cada fila contiene las 10 posiciones de la fila y tiene un puntero hacia la fila siguiente y hacia la anterior. Esto permite agregar y eliminar filas cuando se completan líneas.

La pieza contiene la información de la pieza que se está utilizando, como su tipo, posición, orientación y los bloques que la forman.

La cola se utiliza para guardar las piezas que aparecerán próximamente. De esta forma se puede mantener el orden en el que las piezas deben ir saliendo.

La pila se utiliza para guardar la pieza que el jugador coloca en la opción Hold. Al sacar la pieza de la pila se obtiene la pieza que estaba guardada.

La estructura de replay es una lista doblemente enlazada que guarda diferentes estados de la partida. Cada nodo contiene una copia del tablero, la pieza actual, la pieza Hold, el puntaje y el nivel. Al tener punteros hacia adelante y hacia atrás se puede avanzar o retroceder entre los estados.

También se utiliza una estructura para los eventos programados, donde se guardan acciones que deben ejecutarse durante la partida. Los eventos se mantienen ordenados según el momento en que deben ejecutarse.

2. Complejidad de las operaciones

La complejidad permite saber aproximadamente cuánto aumenta el trabajo que realiza una operación cuando aumenta la cantidad de datos.

Las principales operaciones implementadas tienen las siguientes complejidades:

Estructura / operación	Complejidad
Insertar en la cola	O(1)
Eliminar de la cola	O(1)
Insertar en la pila	O(1)
Eliminar de la pila	O(1)
Avanzar en replay	O(1)

Estructura / operación	Complejidad
Retroceder en replay	$O(1)$
Guardar un estado de replay	$O(1)$
Eliminar estados posteriores en replay	$O(n)$
Limpiar la lista de replay	$O(n)$
Buscar una fila del tablero	$O(n)$
Insertar una fila	$O(1)$
Eliminar una fila ya localizada	$O(1)$
Recorrer las filas del tablero	$O(n)$
Ordenamiento $O(n^2)$	$O(n^2)$
Ordenamiento $O(n \log n)$	$O(n \log n)$

En las operaciones $O(1)$, el tiempo de ejecución no depende directamente de la cantidad de elementos. En cambio, en operaciones como $O(n)$ u $O(n^2)$, el trabajo aumenta conforme aumenta la cantidad de datos.

2. Tabla o gráfico de tiempos de ordenamiento

Registros	Insertion Sort $O(n^2)$	Merge Sort $O(n \log n)$
10	0.000 ms	0.000 ms
100	0.04 ms	0.08 ms
1 000	1.12 ms	0.095 ms
10 000	125 ms	1.35 ms

Respuestas a las preguntas de análisis

1. Al utilizar pocos registros, la diferencia entre los dos algoritmos es pequeña. En este caso, con 100 registros los tiempos son similares. Al aumentar la cantidad de datos, la diferencia se hace más evidente, especialmente desde 1 000 registros. Con 10 000 registros, Insertion Sort tarda mucho más que Merge Sort. Esto coincide con sus complejidades: $O(n^2)$ y $O(n \log n)$, respectivamente.
2. El tablero como lista doblemente enlazada permite eliminar una fila completa sin desplazar las demás. Insertar una fila al inicio y eliminar una fila ya localizada tienen un costo $O(1)$. Buscar una fila puede tener un costo $O(n)$.
3. El replay utiliza una lista doblemente enlazada porque permite avanzar y retroceder entre los estados de la partida. Moverse al estado anterior o siguiente tiene un costo $O(1)$.
4. La cola de eventos mantiene los eventos ordenados por su tiempo de ejecución, dejando al frente el que debe ejecutarse primero. Insertar un evento puede requerir recorrer la lista, por lo que tiene un costo $O(n)$ en el peor caso. Obtener el evento del frente es $O(1)$.